

CS2300: File Structures and Introduction to Database Systems

Disk Storage, Basic File Structures,
Hashing, and Modern Storage
Architectures

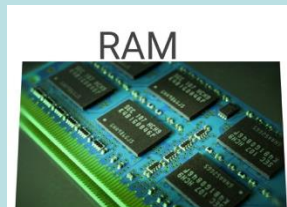
San Yeung

Introduction

Database data must be stored *physically* on some **computer storage medium**

Storage Hierarchy

1. **Primary Storage:** Operated on directly by the computer's CPU, like **main memory** and caches.
> *Random data access*
2. **Secondary Storage:** For storing moderate amounts of permanent data, like **magnetic disks** and SSDs/flash memories.
> *Random data access*
3. **Tertiary Storage:** For offline storage and archiving databases, like optical disks (CDs, DVDs) and **magnetic tapes**.
> *Sequential data access*



The Memory Hierarchy

- Going down the hierarchy:
 - decreasing cost per bit
 - increasing capacity
 - increasing access time
 - decreasing frequency of access to the memory by the processor

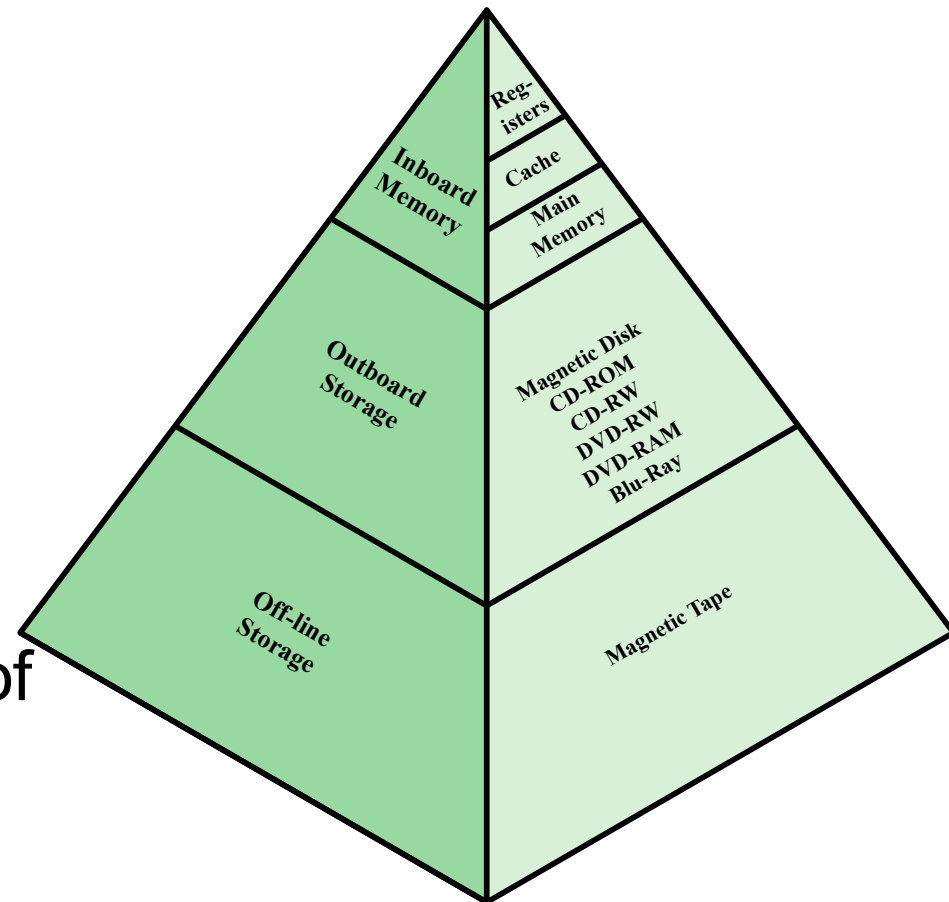


Figure 1.14 The Memory Hierarchy

Storage Organization of Databases

- Most databases are stored permanently on magnetic disk *secondary storage*
 1. Most databases are **too large** to fit entirely in main memory
 2. Main memory is often categorized as **volatile** storage whereas secondary storage is **nonvolatile**
 1. Less frequent of data loss
 3. Secondary storage has a *lower cost* of storage per unit of data

Storage Organization of Databases

- **Physical Database Design:**
 - Choosing the particular data organization techniques that best suit the given application requirements
- **Typical database applications:**
 - Need only a *small* portion of the database at a time for processing
- **When data is needed:**
 1. *Locate* the data on disk
 2. *Copy* the data to main memory for processing
 3. *Rewritten* to the disk if the data is changed
- The data stored on disk is organized as **files of records**
- Each **record** is a collection of data values
 - Entities
 - Attributes
 - Relationships

Storage Organization of Databases

- Records should be stored on disk in a manner that can be **located efficiently** when needed
- **Primary File Organizations:**
 - Determine how the file records are **physically** placed on the disk
 - Which also determine how the records can be accessed
 - How many primary file organization will one database table have?
 - *Heap file* – unordered
 - *Sorted file* – sequential
 - *Hashed file* – uses a hash function
 - *B-tree file* – uses tree structures
- **Secondary File Organizations/ Auxiliary Access Structures:**
 - Allow a more efficient access to records based on *alternate fields*
 - Most exist as **indexes**
 - How many secondary file organization will one database table have?

Characteristics of Magnetic Disk

- **Hard Disk Drive (HDD)**
 - The device that holds the magnetic disks
- The basic unit of data on the disk is a single *bit*
 - A *magnetized* area on disk can represent a bit value (0 or 1)
- The **capacity** of a disk is the number of bytes it can store



- All disks are made of magnetic material shaped as a thin **circular disk**
- A disk can be single-sided or **double-sided** if both surfaces are used to store information
- To increase storage capacity, disks are assembled into a **disk pack**



16.1 (a) A single-sided disk with read/write hardware. (b) A disk pack with read/write hardware

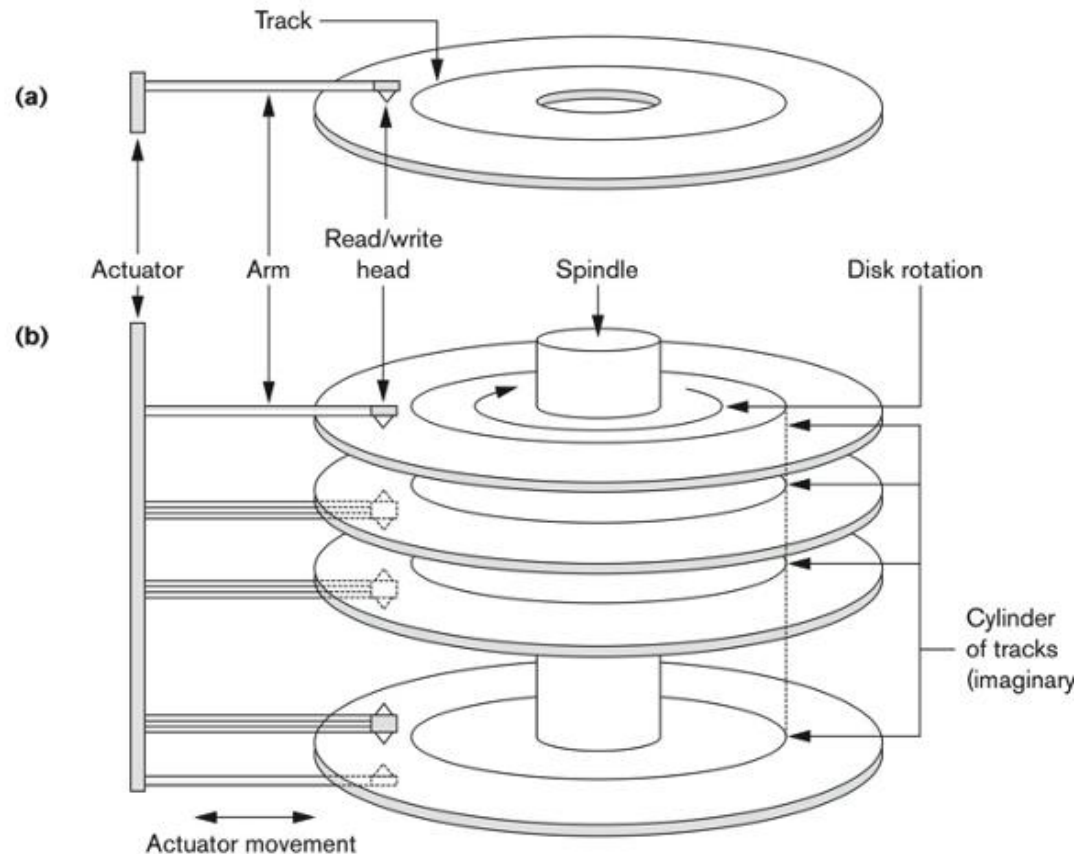
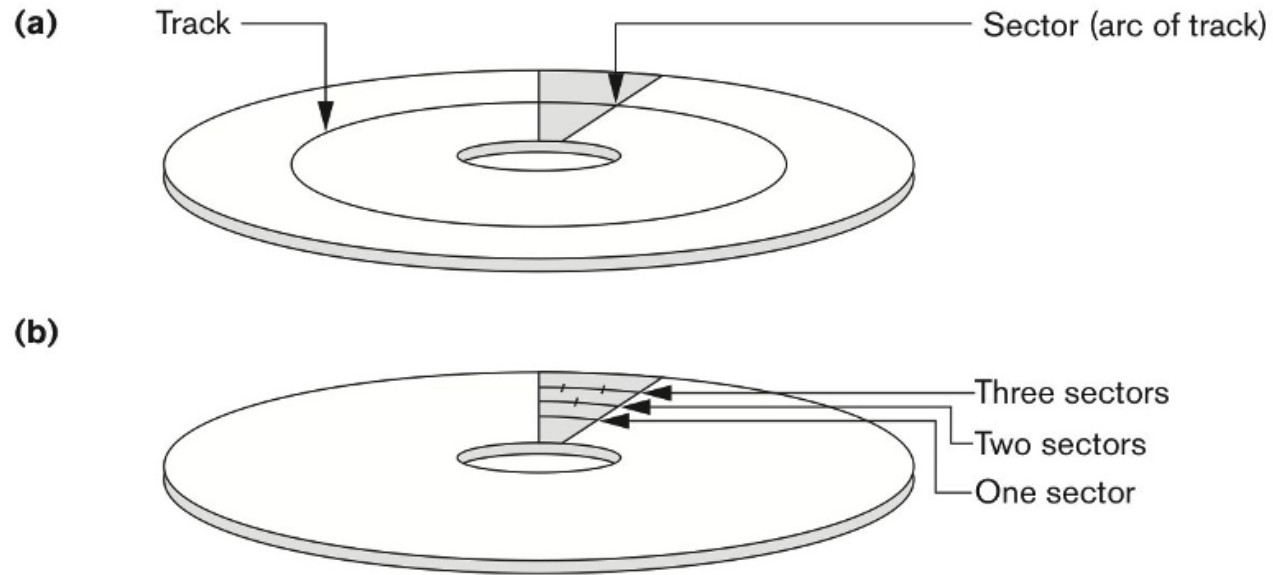
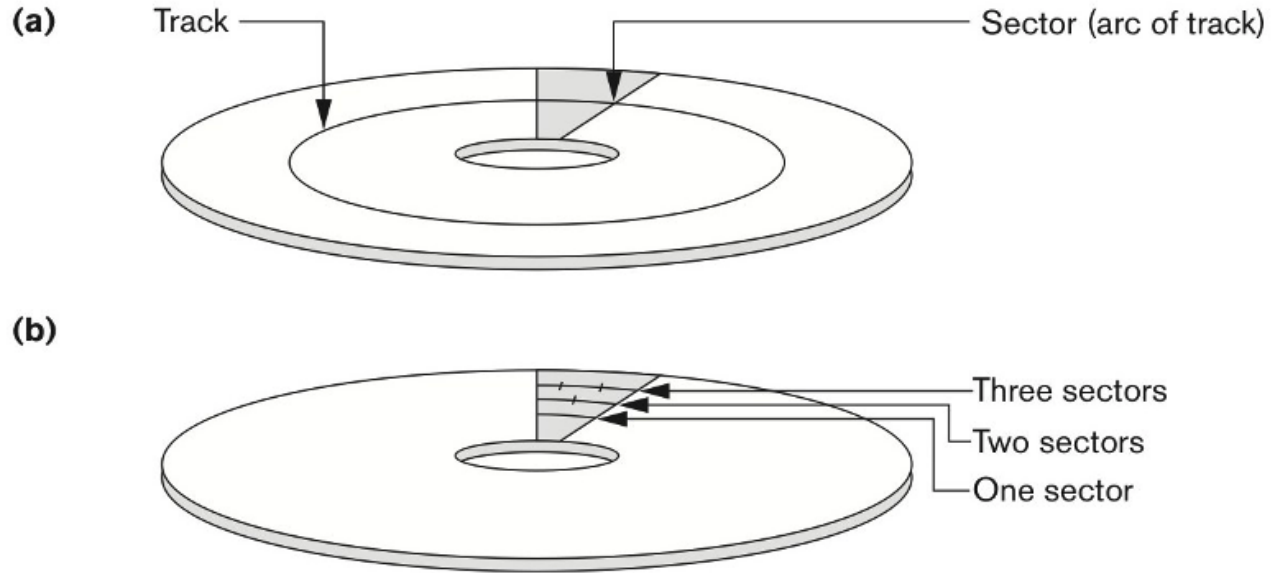


Figure 16.2 Different sector organizations on disk. (a) Sectors subtending a fixed angle. (b) Sectors maintaining a uniform recording density.



- Disks are divided into concentric circular **tracks** on each disk surface
- Tracks with the same diameter on the various surfaces are called a **cylinder**
- Which is faster?
 - Retrieve data stored on **one cylinder**, or
 - Data distributed among **different cylinders**

Figure 16.2 Different sector organizations on disk. (a) Sectors subtending a fixed angle. (b) Sectors maintaining a uniform recording density.

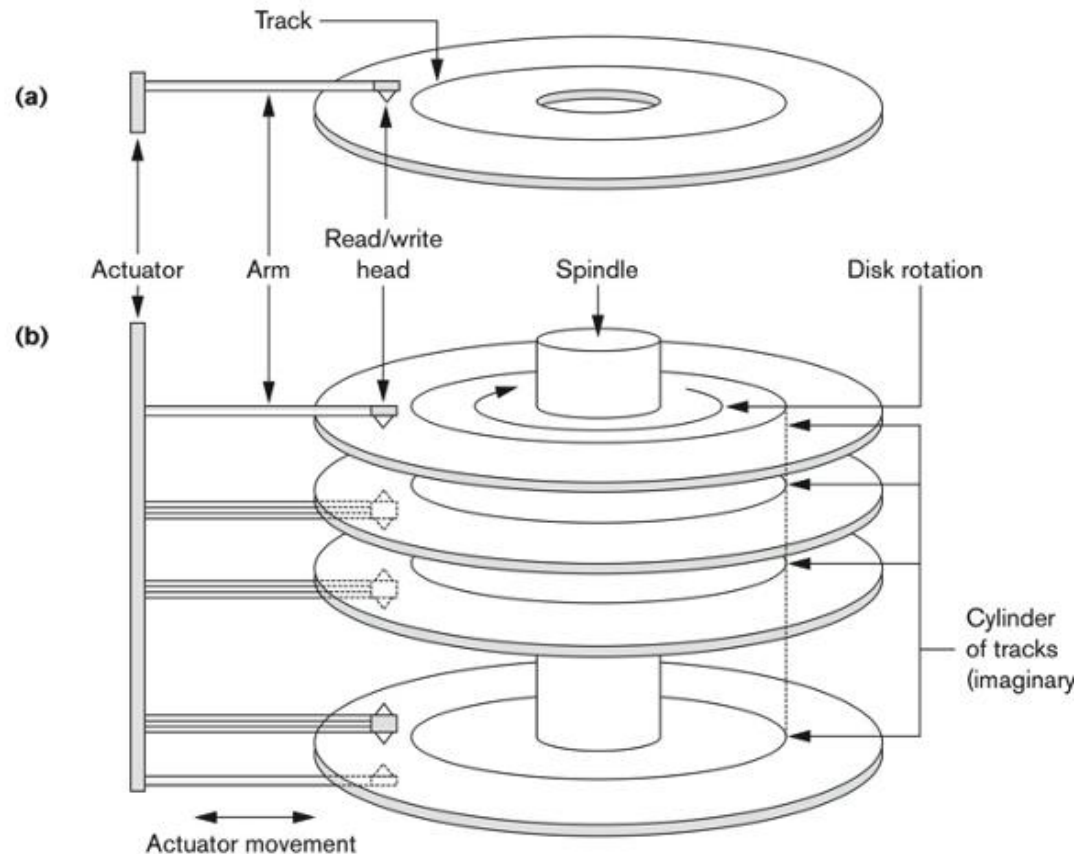


- The number of tracks on a disk ranges from a few thousand to 100,000+
- A track usually is divided into smaller blocks or **sectors**
 - The number is hard-coded on the disk surface and cannot be changed
- A common sector size is 512 bytes (similar to the **disk block** size of a computer system)

- A disk is a **random access** addressable device
- **Transfer** of data between the main memory and disk takes place in units of disk blocks
- Sometimes several contiguous block, called a **cluster**, may be transferred as a unit



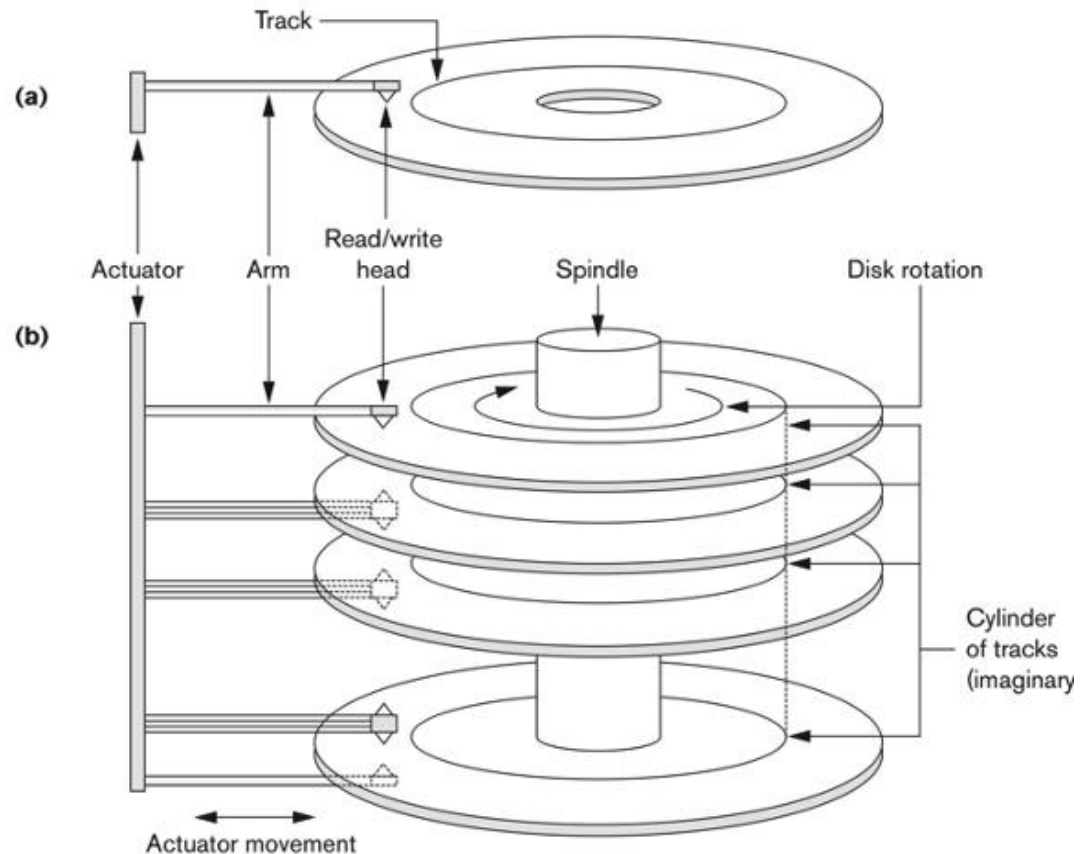
16.1 (a) A single-sided disk with read/write hardware. (b) A disk pack with read/write hardware



- The disk **read/write head** is responsible for reading or writing a block
- It is a part of a system called a **disk drive**
- Disk packs with multiple surfaces are controlled by **several** read/write heads—one for each surface



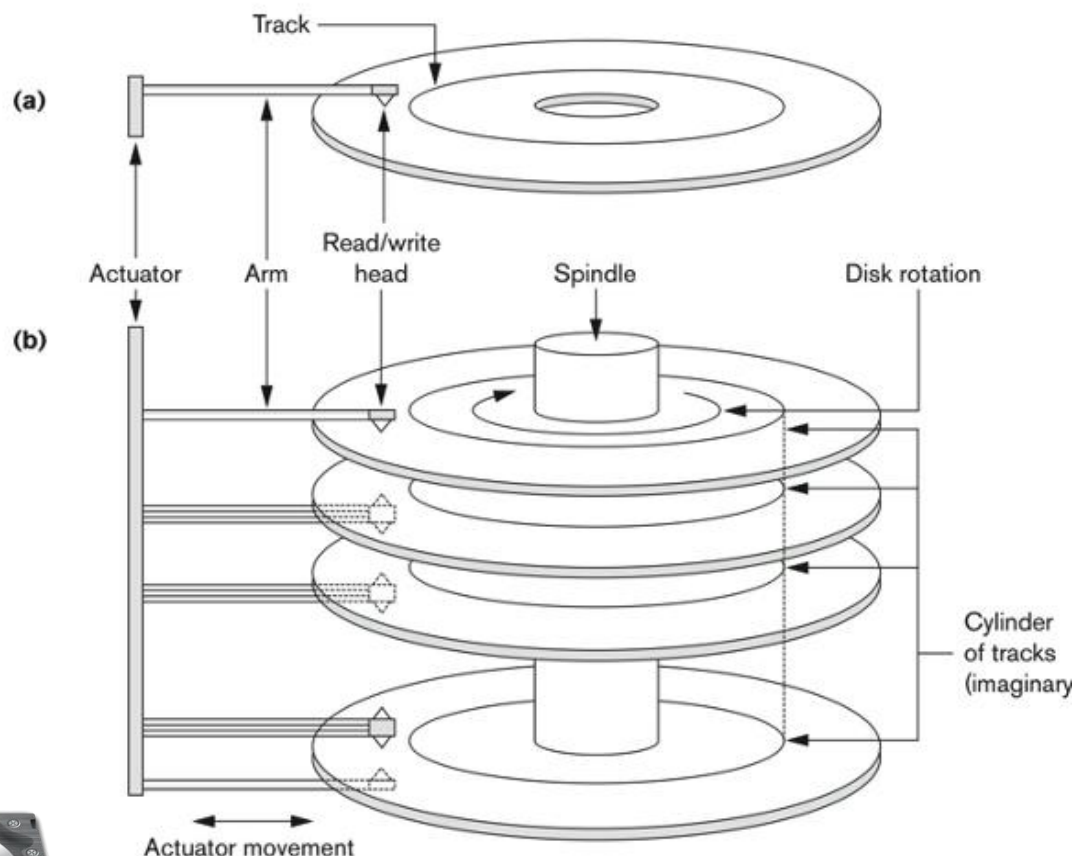
16.1 (a) A single-sided disk with read/write hardware. (b) A disk pack with read/write har



- Disk drives **rotate** the disk pack continuously at a constant speed
 - Typically between 5,400 and 15,000 rpm
- Data transfer
 1. The head needs to be **positioned** on the right track
 2. The specified **block address** needs to be located
 3. *Activate* the read/write head to start transferring data

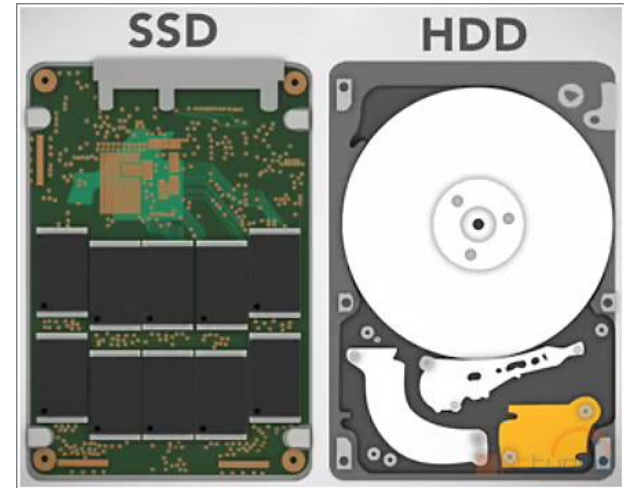


16.1 (a) A single-sided disk with read/write hardware. (b) A disk pack with read/write hardware



Disk Storage Devices

- Reading or writing a disk block is time consuming because of the **seek time** and **rotational delay** (latency).
- Locating data on disk is a major bottleneck in database applications.
 - **HDD + SSD**
- **Physical Hardware Address** (of a block):
 - * Cylinder number +
 - * Track number +
 - * Page number (aka. Logical Block Address)



Files of Records

Fields



Records

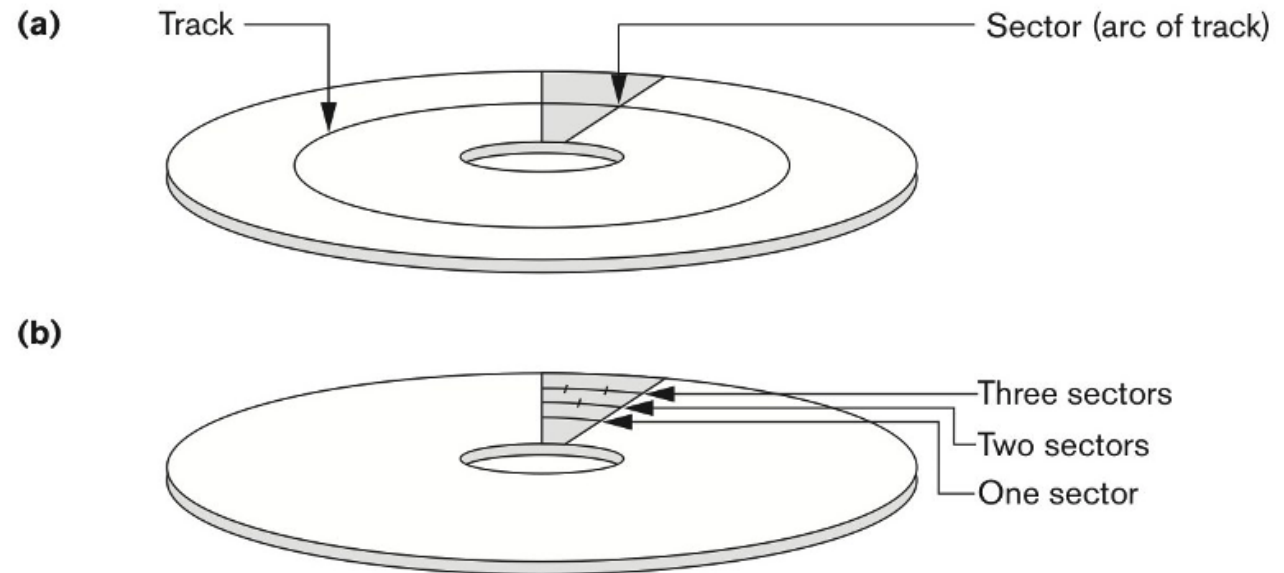


Blocks



Files

Figure 16.2 Different sector organizations on disk. (a) Sectors subtending a fixed angle. (b) Sectors maintaining a uniform recording density.



Files of Records

- A file can have *fixed-length* records or *variable-length* records.

800	J o h n s m i t h	2
-----	---------------------	---

1	T o m	1
---	-------	---

- A record may have *fixed-length* fields or *variable-length* fields.

EID=800	?	Name = John Smith	?	Dept = 2	\$
---------	---	-------------------	---	----------	----

EID=1	?	Name = Tom	?	Dept = 1	\$
-------	---	------------	---	----------	----

Fixed-length Records

Employee record

(1) EID, 2 byte integer

(2) Name, 10 char.

(3) Dept, 2 byte code

Schema

800	J o h n s m i t h	2
-----	---------------------	---

1	T o m	1
---	-------	---

Records

Variable-length Records

- For variable-length fields, we do not know the exact length of some field values.
- We use special separator characters such as ?, % or \$.

EID=800	?	Name = John Smith	?	Dept = 2	\$
---------	---	-------------------	---	----------	----

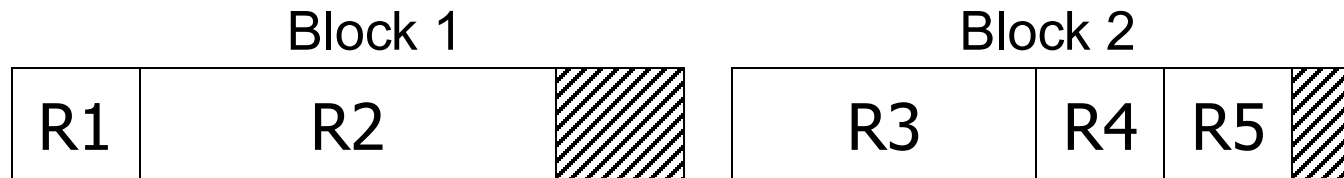
EID=1	?	Name = Tom	?	Dept = 1	\$
-------	---	------------	---	----------	----

Record Blocking

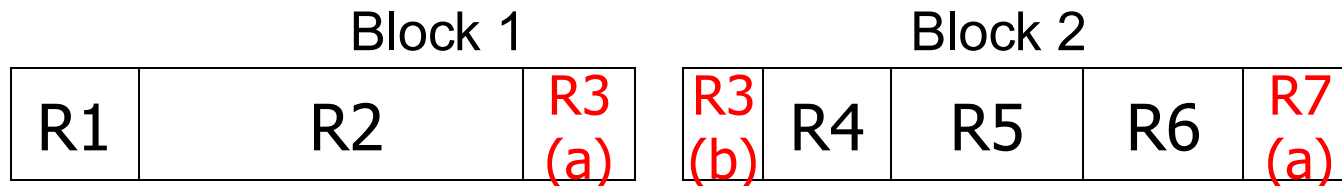
- The *records* of a file **must** be *allocated* to disk blocks
 - A block is the unit of *data transfer* between disk and memory
- Case 1) block size is *larger* then the record size
 - Then each block will contain >1 records
- Case 2) some files may have unusually large records that cannot fit in one block
- Let the block size be B bytes and the file size R bytes
- With $B \geq R$, we can fit $\lfloor B/R \rfloor$ records per block $\leftarrow$ this is referred to the **blocking factor** (bfr) for the file
- If there are some unused space in a block, it can be computed as:
 - $B - (bfr * R)$ bytes

Spanned vs. Unspanned Records

- Records can be *unspanned* (no record can span two blocks) or *spanned* (a record can be stored in more than one block).
- Unspanned

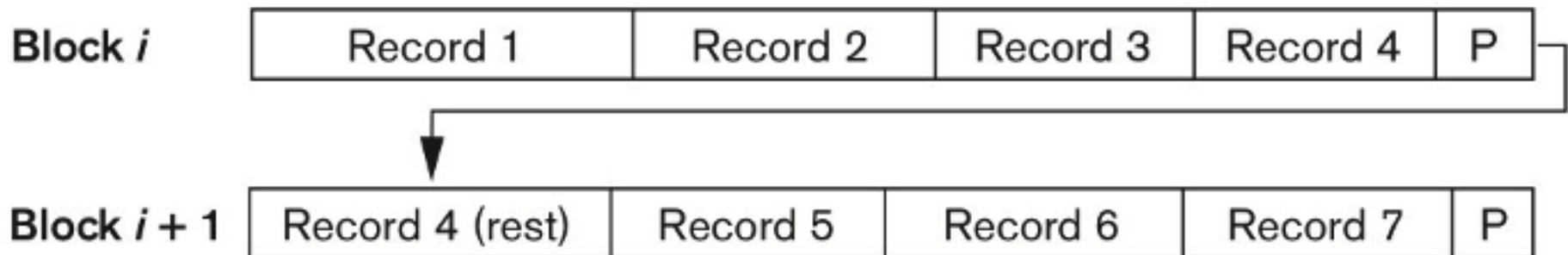


- Spanned



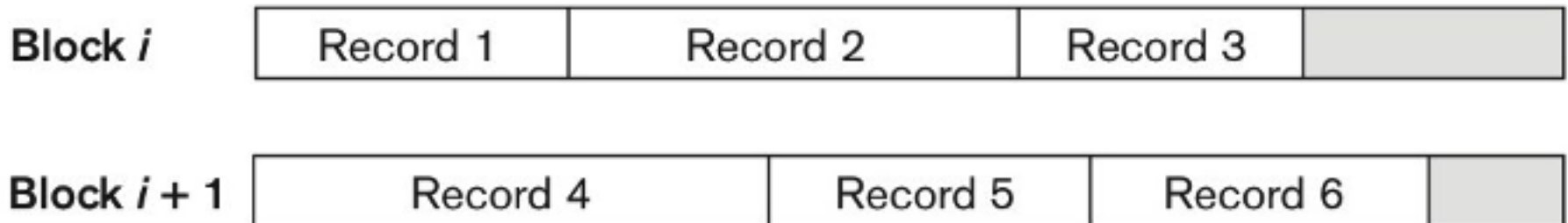
Spanned Records

- Spanned: records can span more than one block
 - Must** be used whenever a record is **larger** than a block
 - A **pointer** at the end of the first block should point to the block containing the remainder of the record
 - do they have to be **consecutive block** on disk?*



Unspanned Records

- Unspanned: records are not allowed to *cross* block boundaries
 - Used with fixed-length records having $B > R$
 - *Simplify* record processing because it's now possible to make each record start at a **known location** in the block
 - For variable-length records, either a spanned or an unspanned organization can be used



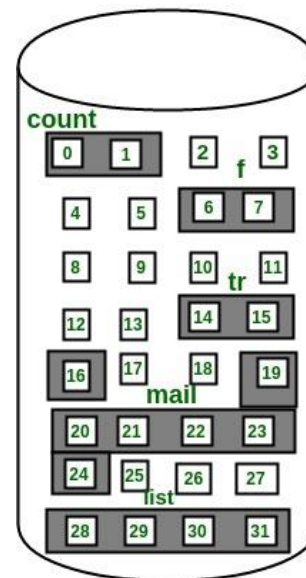


Allocating File Blocks on Disk

- Several standard techniques:
 - Contiguous allocation
 - Linked allocation
 - Linked Contiguous allocation (a combination)
 - Indexed allocation
 - Hybrid of any of the above

Allocating File Blocks on Disk

- Contiguous allocation
 - file blocks are allocated to consecutive disk blocks
 - *Same track or same disk cylinder*
 - This makes reading the whole file very _____?
 - Especially if I/O buffering is equipped
 - makes expanding the file _____ ?

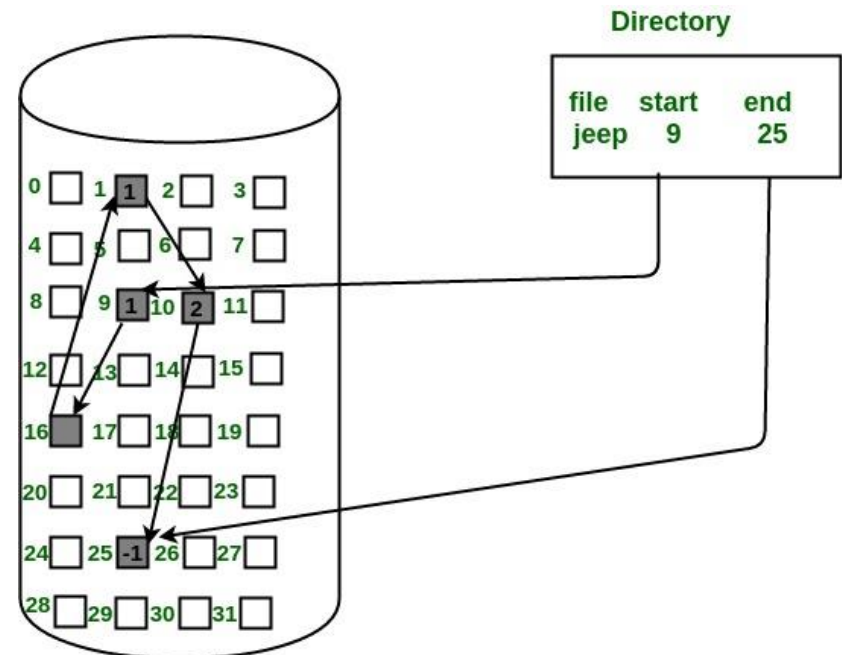


Directory

file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Allocating File Blocks on Disk

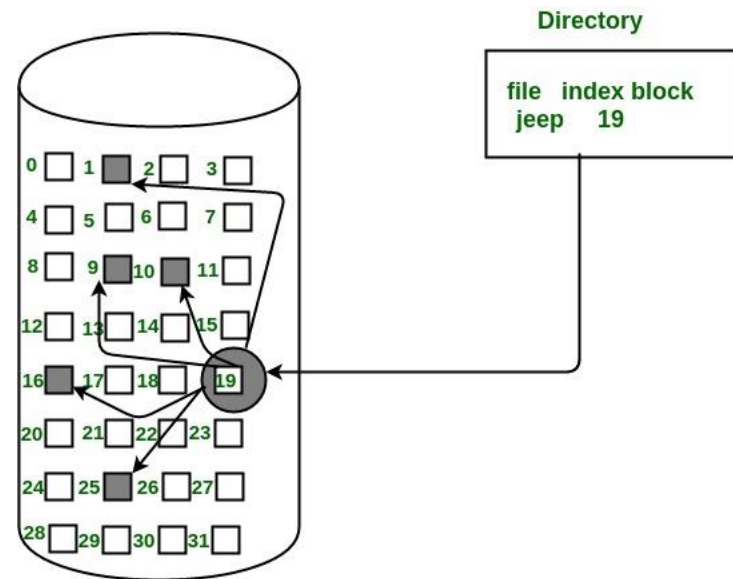
- Linked allocation
 - each file block contains a **pointer** to the next file block
 - makes it _____ to expand but _____ to read the whole file
 - *Scattered tracks and disk cylinders*



Allocating File Blocks on Disk

- Combination of “Linked” and “Contiguous”
 - Allocates **clusters** of consecutive disk blocks
 - Also known as *file segments* or *extents*
 - Maintains linkage among the clusters

- Indexed allocation
 - Maintains **index blocks** which in turn contain *pointers* to the actual file blocks
 - Each entry in the index block **directly** points to a specific block where part of the file's data is stored
 - Supports **direct access** to the blocks of interests



File Headers

- A **file header** or file descriptor contains information about a file that is needed by the **system programs** that access the file records
 - E.g. record format descriptions and field lengths
- To **search** for a record on disk:
 - Block address is known: simply retrieve the block and copy its data into main memory
 - Block address is *unknown*:
 - search programs must do a **linear search** through the file blocks until the record is located or all blocks have been searched unsuccessfully
- **Goal of a good file organization:**
 - to avoid full scan of the file and to locate the block with a minimal number of block transfers

Operations on Files

Actual operations vary from system to system

- OPEN
- FIND
- FINDNEXT
- READ
- INSERT
- DELETE
- MODIFY
- CLOSE

File Organization vs Access Methods

- File organization:
 - the *organization* of the data of a file into records, blocks, and access structures
 - Dictate how records and blocks are stored on the storage medium and *interlinked*
- Access method:
 - provides a group of operations that can be applied to a file
 - In general, it is possible to apply several access methods to a file
 - Each file is only organized using a certain organization
 - Some access methods are dependent on how files are organized in certain ways

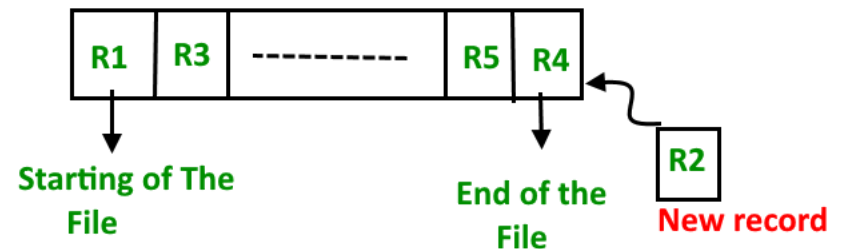


Outline

- Disk storage devices
- Files of records
- Operations on files
- Types of files (File Organizations)
 - Unordered files
 - Ordered files
 - Hash files

Files of Unordered Records (Heap Files)

- This is the **simplest** and most basic type of file organization
 - Records are placed in the file in the order in which they are inserted
 - *Where do you insert **new records**?*
- Common names for this organization:
 - Heap file
 - Pile file



Files of Unordered Records (Heap Files)

- Inserting a new record is very *efficient*
 - The new record is added to the **last** disk block of the file
 - The address of the last file block is kept in the *file header* after the block is rewritten back to disk
- To **search** for a record, it is expensive
 - Why?
 - a ***linear search*** through the file records is necessary
 - On the average, a program will need to search **half** the file blocks if trying to locate a single record

Files of Unordered Records (Heap Files)

- **Deleting** a record
 - 1) delete the record from the file block
 - This leaves *unused* space in the disk blocks
 - 2) Have an extra bit to represent the **deletion marker** for each record
 - A record is deleted by setting the marker to a certain value
- *Both* deletion techniques require periodic **reorganization** of the file to reclaim/reorganize the unused space of deleted records
- **Sorting can be an expensive operation!**



Files of Ordered Records (Sorted Files)

- Also called a *sorted, ordered, or sequential file*.
- File records are kept sorted by the values of an *ordering field*.
- If the ordering field is also a **key field** of the file, then the field is called the **ordering key** for the file
 - The key guaranteed that each record will have a unique value

Files of Ordered Records (Sorted Files)

	Name	Ssn	Birth_date	Job	Salary	Sex
Block 1	Aaron, Ed					
	Abbott, Diane					
	Acosta, Marc					
Block 2	Adams, John					
	Adams, Robin					
	Akers, Jan					
Block 3	Alexander, Ed					
	Alfred, Bob					
	Allen, Sam					

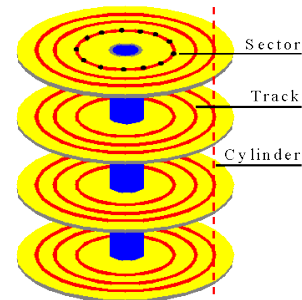
- An example sequential file of EMPLOYEE records
- Which attribute field is used as the *ordering key field*?
- If the ordering attribute is not a key attribute → *Ordered Clustered File*

Files of Ordered Records (Sorted Files)

- Advantages:
 - **reading** the records in order of the ordering key values becomes extremely efficient
 - No need for additional sorting
 - finding the *next* record is usually easy because it is in the same block as the current one
 - **Search** based on the value of an ordering key field results in faster access when applying the _____ search technique
- What might be some of the **disadvantages**?

Files of Ordered Records (Sorted Files)

- Disadvantages:
 - **Insertion** is **expensive**: records must be inserted in the *correct order*.
 - On average, **half** the records of the file must be moved to make space for the new record
 - **Deletion** is less severe of a problem if deletion markers and periodic reorganization are used
- Is this true?
 - Ordered file blocks are stored on contiguous cylinders to minimize the seek time.

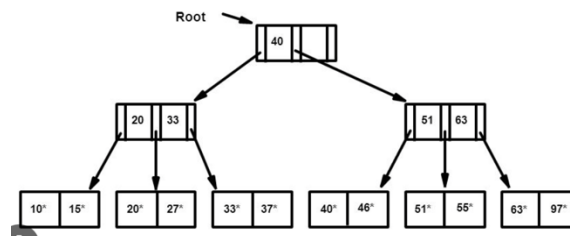


Ordered Files

Table 16.3 Average Access Times for a File of b Blocks under Basic File Organizations

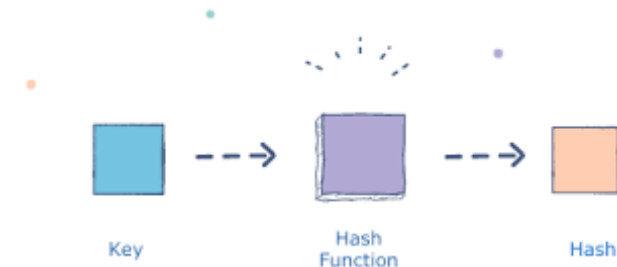
Type of Organization	Access/Search Method	Average Blocks to Access a Specific Record
Heap (unordered)	Sequential scan (linear search)	$b/2$
Ordered	Sequential scan	$b/2$
Ordered	Binary search	$\log_2 b$

- Often requires a **primary index** to be practical
- **Indexed-Sequential File**
- **Indexed-Sequential Access Method (ISAM)**



Hashing Techniques for Files

- Hashing is another type of *primary file organization*
- Provides very **fast** access to records under certain search conditions
 - The condition must be an **equality** on a single field (the **hash field**)
- Main idea of hashing:
 - Provide a **hash function** to the hash field value of a record and yields the **address** of the disk block in which the record is stored



Hashing for Disk Files

- Hashing for disk files is called **external hashing**
- **Static external hashing:**
 - The designated *target address space* for the file is made of **buckets**
 - The number of buckets is **fixed/static**
 - A bucket is either one disk block **or** a cluster of contiguous disk blocks
 - The hashing function maps a key into a *relative bucket number*
 - A table maintained in the file header **converts** the bucket number into the corresponding disk block address

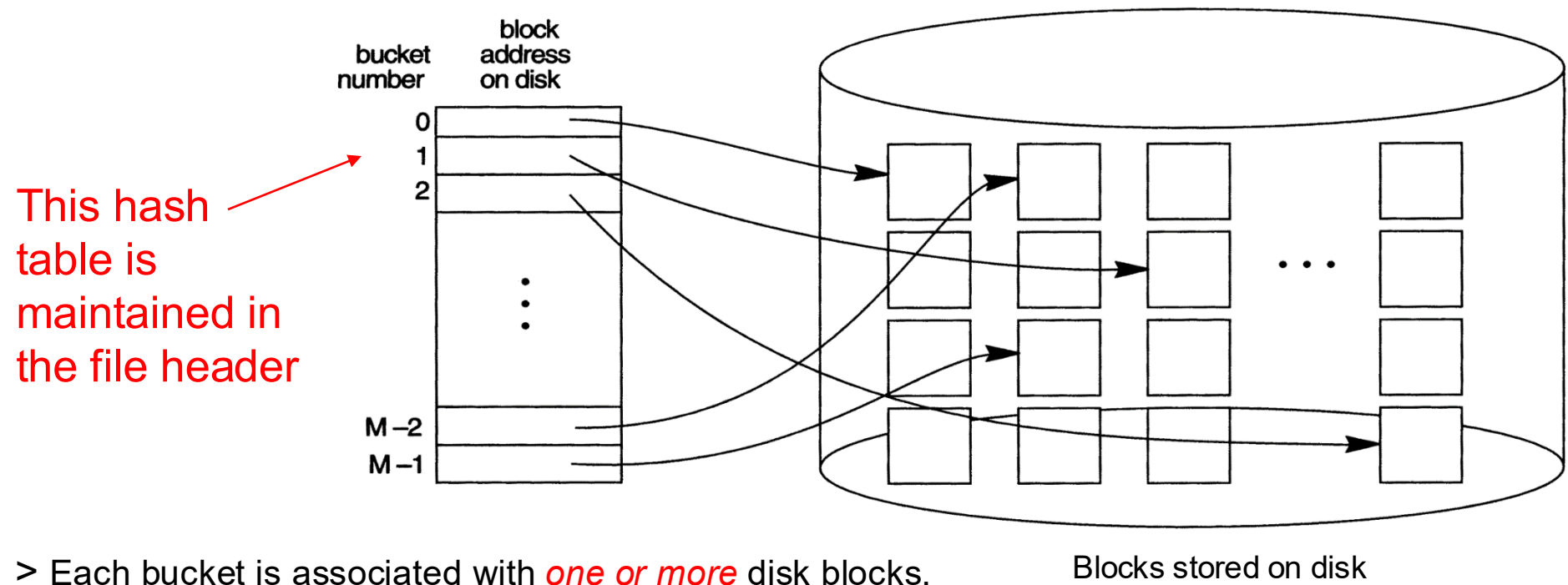
Example of a Hash File

- There are 10 buckets.
- Suppose the hash function on the *hash* key returns the following values:
 - $h(\text{Perryridge}) = 5$
 - $h(\text{Round Hill}) = 3$
 - $h(\text{Brighton}) = 3$
- **Collisions?**
 - occur when a new record hashes to a bucket that is already full.

bucket 0			bucket 5		
			A-102	Perryridge	400
			A-201	Perryridge	900
			A-218	Perryridge	700
bucket 1			bucket 6		
bucket 2			bucket 7		
			A-215	Mianus	700
bucket 3			bucket 8		
A-217	Brighton	750	A-101	Downtown	500
A-305	Round Hill	350	A-110	Downtown	600
bucket 4			bucket 9		
A-222	Redwood	700			

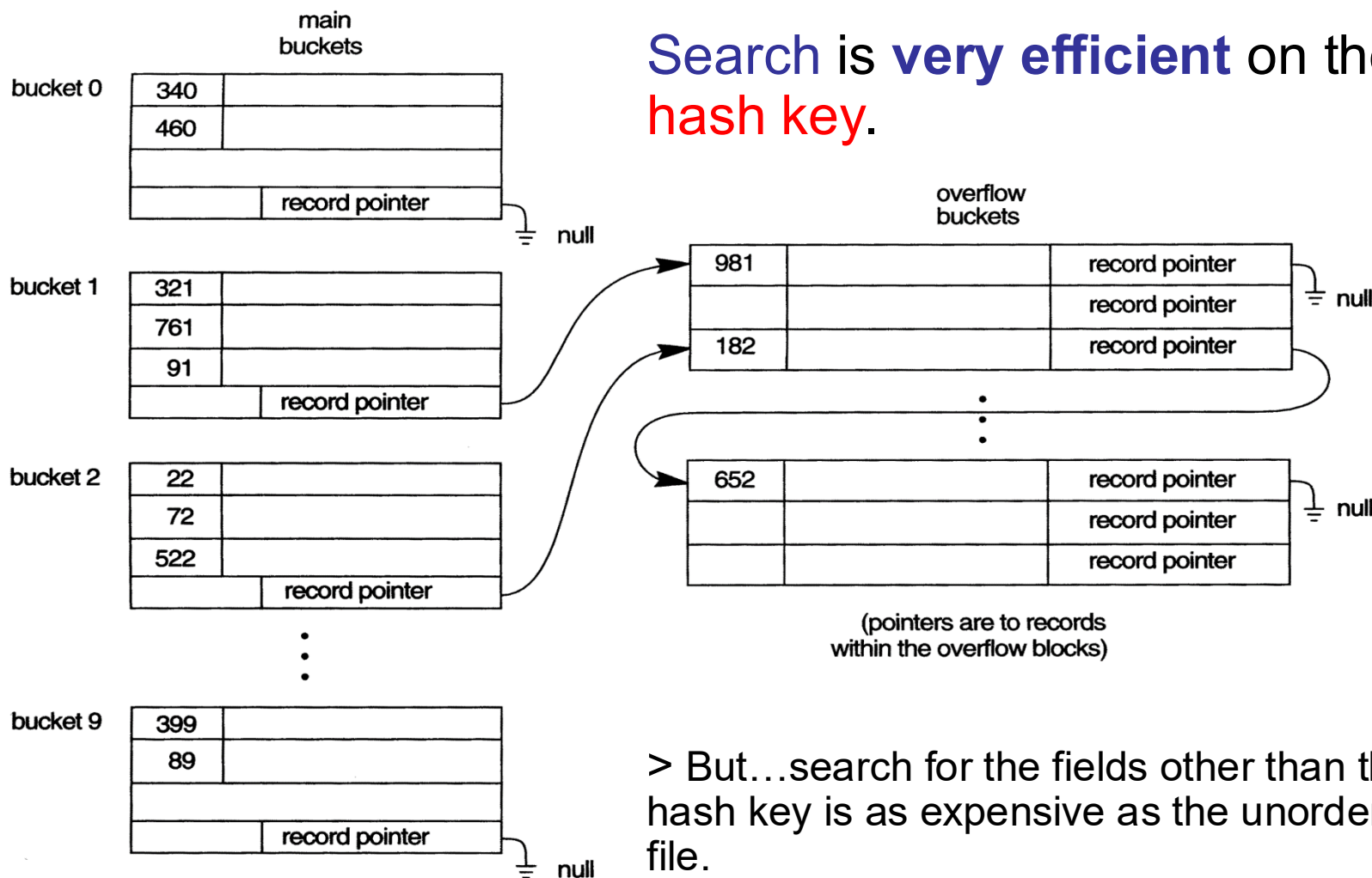
Static External Hashing

- The record with hash key value K is stored in bucket_i , where $i = h(K)$, and h is the *hashing function*.



Hashed Files - Overflow handling

Search is **very efficient** on the **hash key**.



> But...search for the fields other than the hash key is as expensive as the unordered file.

Static External Hashing

- Static hashing:
 - A **fixed** number of buckets is used
 - i.e. the address space is fixed
 - Could be a serious drawback for **dynamic files**
- Shrinking a file
 - To avoid a lot of *unused* space in the hash buckets
- Expanding a file
 - Could lead to *high collision* and slow down retrieval because of the *long lists* of overflow records
- A solution:
 - Physically change the number of buckets
 - This calls for a **new** hashing function to **redistribute** the records
 - Will the rehashing/reorganization be time consuming, especially if working with large files?



Other Primary File Organizations

- Files of Mixed Records
 - *assign an area of the disk to hold records of more than one type so they can be **physically clustered** on disk*
 - *object DBMSs often store related objects together in a mixed file*
- B-Trees and Other Data Structures
 - B-tree
 - if both the record size and the number of records in a file are small
 - *In general, **any** data structure that can be adapted to the characteristics of disk devices can be used as a primary file organization for 'record placement' on disk*
 - Column-based storage of data (vs row-based)
 - relational databases
 - great for performing aggregate analysis on BIG data

Indexes

A database index is similar to the index in a book, which allows quick location of information (data records) without reading the entire book (data file).

DB indexes **complement** *primary file organization* techniques by providing an additional layer of optimization that enhances data retrieval performance.

- * Auxiliary Data File Access Structures.
- * Provide **secondary access paths**.

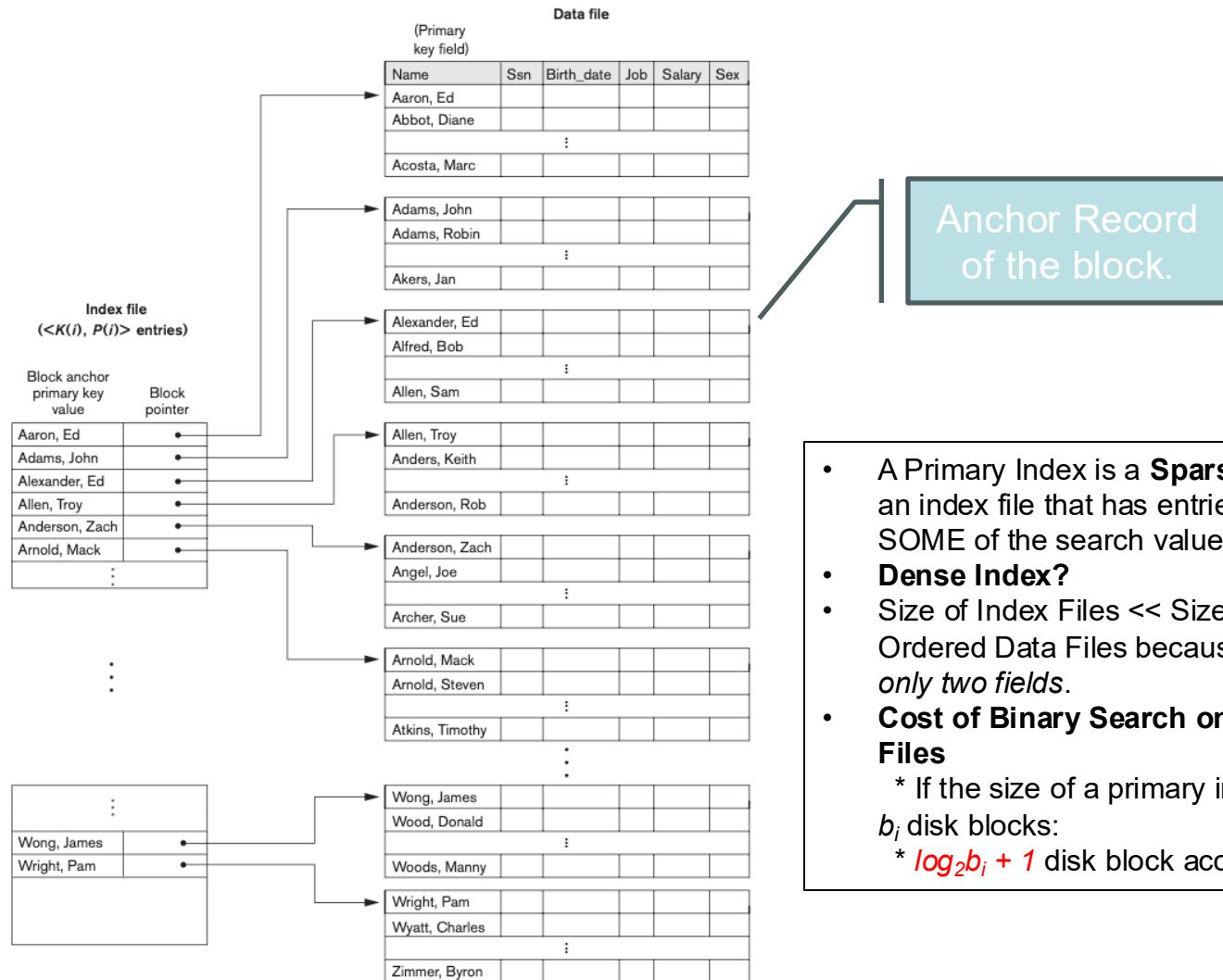
Trade-Offs: **Space Overhead** (more indexes = additional files), **Maintenance Overhead** (when do indexes need to be updated?). **Balancing** between *over-indexing* and *under-indexing*.

Primary Indexes

Primary Index: An ordered file whose records are with two fields.

- * Acts as an access structure to efficiently search for and access data records in a **sorted data file** using its *primary key*.
- * $\langle K(i), P(i) \rangle$
- * $K(i)$ is the **primary key** value of index entry i
- * $P(i)$ is the pointer to a disk block of index entry i
- * The total number of entries in the index is the SAME as the number of disk blocks in the ordered data file.

Flat table, hash table, and search tree are the techniques that can be used to build either a primary or a secondary index.



Primary Index is an indexing of a key attribute of a physically ordered data file. Sparse.

Primary (Ordered) Indexes

1		1 Class II	20120926	20121003	90021-161	20120926	20120921	F-2396-20	Los Angele	63150	California	M & K Trac	Voluntary:
6		2 Class I	20120924	20121003	91706-216	20121025	20120831	F-2382-20	Irwindale	63062	Nationwid	Ready Pac	Voluntary:
11		3 Class II	20120927	20121003	92833-251	20121217	20120712	F-2438-20	Fullerton	62991	Nationwid	Pulmuone	Voluntary:
16		4 Class II	20120927	20121003	92833-251	20121217	20120712	F-2436-20	Fullerton	62991	Nationwid	Pulmuone	Voluntary:
...		5 Class II	20120921	20121003	17545-963	20130716	20120417	F-2374-20	Manheim	62465	Product w:	Haldeman	Voluntary:
		6 Class I	20120921	20121003	92056-585	20121119	20120824	F-2368-20	Oceanside	63002	nationwid	MyoSci Te	
		7 Class I	20120924	20121003	49548-563	20121204	20120830	F-2390-20	Wyoming	63058	restaurant	Gordon Fo	
		8 Class II	20120928	20121010	17529-976	20131018	20120828	F-2447-20	Gordonvill	63055	Distributio	Sadie's Sal	
		9 Class I	20121001	20121010	53511-955	20121220	20120831	F-0007-20	Beloit	63084	The produ	Kerry Foo	
		10 Class I	20120928	20121010	17529-976	20131018	20120828	F-1074-20	Gordonvill	63055	Distributio	Sadie's Sal	
		11 Class II	20120927	20121003	17527-969	20140811	20111115	F-2401-20	Gap	60642	Product w:	McClures,	
		12 Class II	20120927	20121003	17527-969	20140811	20111115	F-2423-20	Gap	60642	Product w:	McClures,	
		13 Class I	20120924	20121003	91706-216	20121025	20120831	F-2387-20	Irwindale	63062	Nationwid	Ready Pac	
		14 Class II	20120927	20121003	17527-969	20140811	20111115	F-2407-20	Gap	60642	Product w:	McClures,	
		15 Class II	20120927	20121003	17527-969	20140811	20111115	F-2413-20	Gap	60642	Product w:	McClures,	
		16 Class I	20120927	20121003	17527-969	20140811	20111115	F-2398-20	Gap	60642	Product w:	McClures,	
		17 Class II	20120927	20121003	92833-251	20121217	20120712	F-2439-20	Fullerton	62991	Nationwid	Pulmuone	
		18 Class I	20121002	20121010	08085-175	20150513	20120830	F-0017-20	Swedesbo	63063	One sole v	Taylor Fari	

Hash Indexes

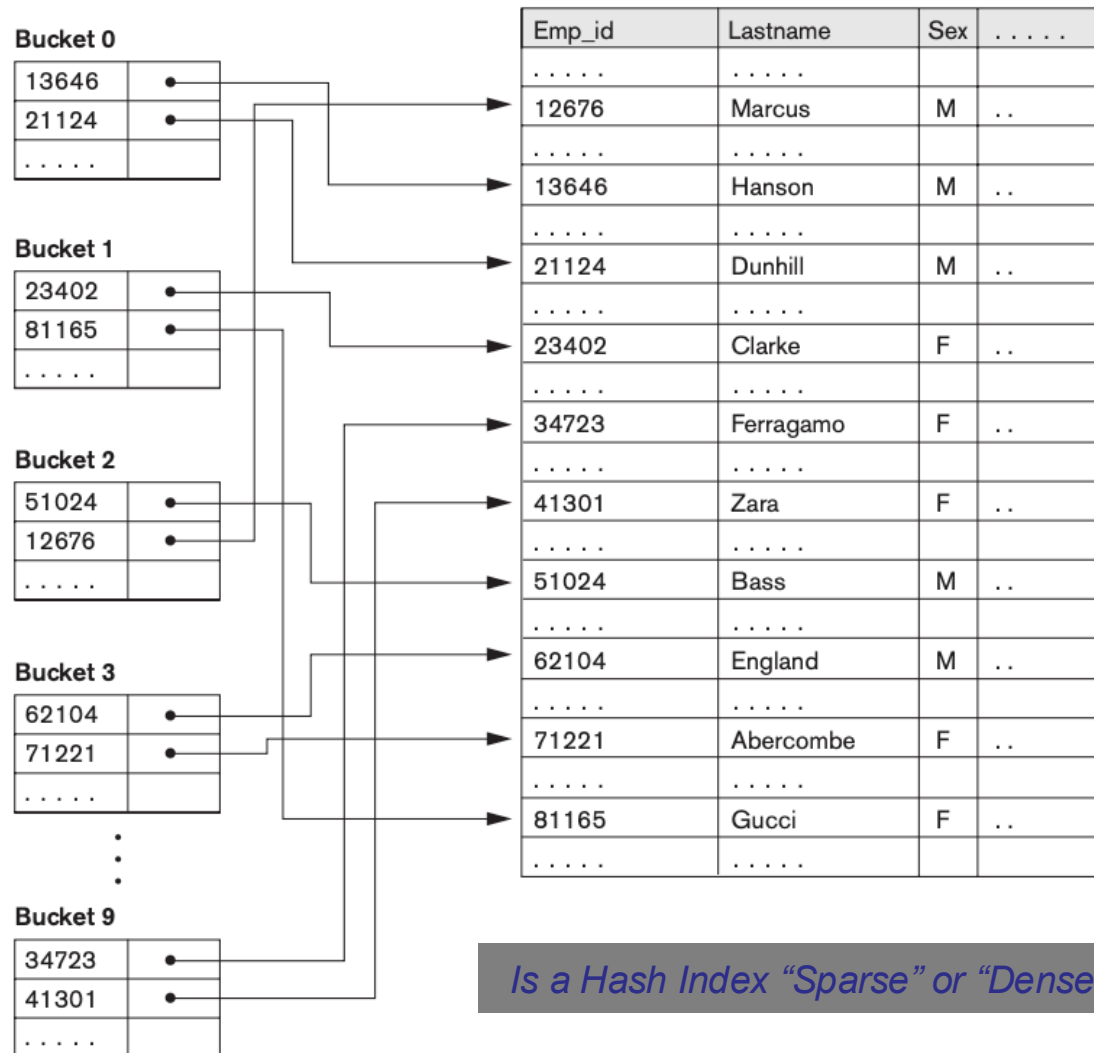
An index created based on *hashing* with index entries of type $\langle K, P \rangle$.

A Hash Index is a secondary structure to access the data file by using hashing on a search key **OTHER** than the one used for the primary data file organization.

Not the primary choice as a primary method of organizing data files
a. less suitable for range queries or sorted retrievals, which are common requirements in database systems.

A primary hash index is possible.

Each index entry contains either a pointer to the record containing the key, or a pointer to the disk block containing the record for that key.



Is a Hash Index "Sparse" or "Dense"?

Hash Indexes

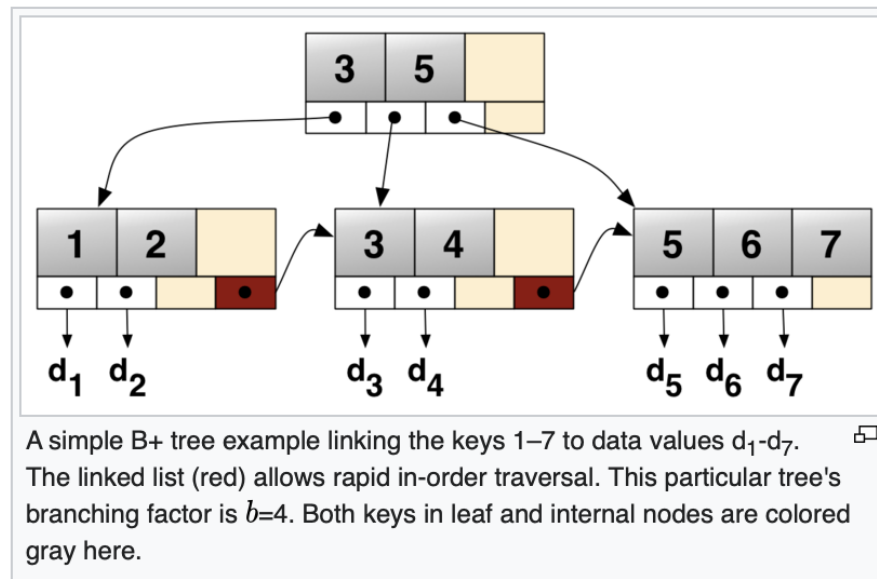
16	
7	
18	
12	
6	
2	
9	
14	
10	
13	
11	
15	

1	Class II	20120926	20121003	90021-161	20120926	20120921	F-2396-20	Los Angele	63150	California	M & K Trac	Voluntary:
2	Class I	20120924	20121003	91706-216	20121025	20120831	F-2382-20	Irwindale	63062	Nationwid	Ready Pac	Voluntary:
3	Class II	20120927	20121003	92833-251	20121217	20120712	F-2438-20	Fullerton	62991	Nationwid	Pulmuone	Voluntary:
4	Class II	20120927	20121003	92833-251	20121217	20120712	F-2436-20	Fullerton	62991	Nationwid	Pulmuone	Voluntary:
5	Class II	20120921	20121003	17545-963	20130716	20120417	F-2374-20	Manheim	62465	Product w:	Haldeman	Voluntary:

6	Class I	20120921	20121003	92056-585	20121119	20120824	F-2368-20	Oceanside	63002	nationwid	MyoSci Te	
7	Class I	20120924	20121003	49548-563	20121204	20120830	F-2390-20	Wyoming	63058	restaurant	Gordon Fo	
8	Class II	20120928	20121010	17529-976	20131018	20120828	F-2447-20	Gordonvill	63055	Distributio	Sadie's Sal	
9	Class I	20121001	20121010	53511-955	20121220	20120831	F-0007-20	Beloit	63084	The produ	Kerry Foo	
10	Class I	20120928	20121010	17529-976	20131018	20120828	F-1074-20	Gordonvill	63055	Distributio	Sadie's Sal	

11	Class II	20120927	20121003	17527-969	20140811	20111115	F-2401-20	Gap	60642	Product w:	McClures,	
12	Class II	20120927	20121003	17527-969	20140811	20111115	F-2423-20	Gap	60642	Product w:	McClures,	
13	Class I	20120924	20121003	91706-216	20121025	20120831	F-2387-20	Irwindale	63062	Nationwid	Ready Pac	
14	Class II	20120927	20121003	17527-969	20140811	20111115	F-2407-20	Gap	60642	Product w:	McClures,	
15	Class II	20120927	20121003	17527-969	20140811	20111115	F-2413-20	Gap	60642	Product w:	McClures,	
16	Class I	20120927	20121003	17527-969	20140811	20111115	F-2398-20	Gap	60642	Product w:	McClures,	
17	Class II	20120927	20121003	92833-251	20121217	20120712	F-2439-20	Fullerton	62991	Nationwid	Pulmuone	
18	Class I	20121002	20121010	08085-175	20150513	20120830	F-0017-20	Swedesbo	63063	One sole v	Taylor Fari	

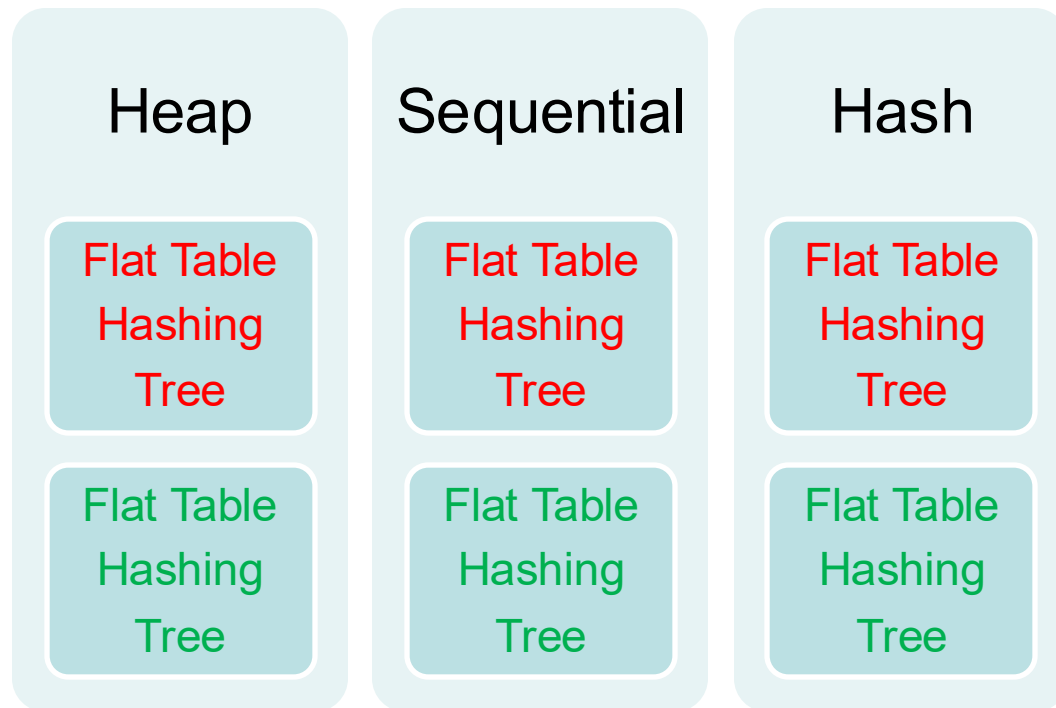
B+-Tree Index: A multilevel balanced-search-tree-based indexing structure where all leaf nodes (interlinked with a link list) signify pointers to the actual data records.



Secondary Indexes

A **secondary index** is an index on one or more columns other than the primary key. A table can have multiple secondary indexes.

Recall that a **primary index** is an index built based on the *primary key* of the relation table. Flat table, hash table, and search tree are the techniques that can be used to build either a primary or a secondary index.



Clustering Index

Clustering Index is an indexing of a (often) non-key attribute. Sparse.

Records are **physically** sorted.

Class I	2	20120924	20121003	91706-216	20121025	20120831	F-2382-20	Irwindale	63062	Nationwid
Class I	6	20120921	20121003	92056-585	20121119	20120824	F-2368-20	Oceanside	63002	nationwid
Class I	7	20120924	20121003	49548-563	20121204	20120830	F-2390-20	Wyoming	63058	restaurant
Class I	9	20121001	20121010	53511-955	20121220	20120831	F-0007-20	Beloit	63084	The produ
Class I	10	20120928	20121010	17529-976	20131018	20120828	F-1074-20	Gordonvill	63055	Distributio
Class I	13	20120924	20121003	91706-216	20121025	20120831	F-2387-20	Irwindale	63062	Nationwid
Class I	16	20120927	20121003	17527-969	20140811	20111115	F-2398-20	Gap	60642	Product w
Class I	18	20121002	20121010	08085-175	20150513	20120830	F-0017-20	Swedesbo	63063	One sole v
Class II	1	20120926	20121003	90021-161	20120926	20120921	F-2396-20	Los Angele	63150	California M & K Trac
Class II	3	20120927	20121003	92833-251	20121217	20120712	F-2438-20	Fullerton	62991	Nationwid Pulmuone
Class II	4	20120927	20121003	92833-251	20121217	20120712	F-2436-20	Fullerton	62991	Nationwid Pulmuone
Class II	5	20120921	20121003	17545-963	20130716	20120417	F-2374-20	Manheim	62465	Product w. Haldeman
Class II	8	20120928	20121010	17529-976	20131018	20120828	F-2447-20	Gordonvill	63055	Distributio Sadie's Sal
Class II	12	20120927	20121003	17527-969	20140811	20111115	F-2423-20	Gap	60642	Product w. McClures,
Class II	14	20120927	20121003	17527-969	20140811	20111115	F-2407-20	Gap	60642	Product w. McClures,
Class II	15	20120927	20121003	17527-969	20140811	20111115	F-2413-20	Gap	60642	Product w. McClures,
Class II	17	20120927	20121003	92833-251	20121217	20120712	F-2439-20	Fullerton	62991	Nationwid Pulmuone
Class II	19	20121001	20121010	33147-652	20141106	20110601	F-0005-20	Miami	63018	FL, GA, IL, Ocho Rios

T/F: If the data in the original file is not already sorted by the clustering key, then creating a clustering index will involve reordering the data or creating a new copy of the database file.

Bitmap Indexes

Bitmap Indexing: a special kind of indexing that uses bitmaps (or bit vectors) on columns that have a low cardinality (i.e., limited number of distinct values). They can be very space efficient and combined (AND-ed, OR-ed) very quickly.

- bitmaps are **compact**, resulting in a much smaller dataset size

Example) Imagine you have a column named `Color` in a table with millions of rows, and the domain of this attribute are of the distinct values: *Red, Blue, and Green*

Relation `Flower`		
Name	Color	Price
Rose	Red	10
Tulip	Blue	15
Daisy	Blue	5
Sunflower	Green	30

Bitmap Index	Red	Blue	Green
Row 1	1	0	0
Row 2	0	1	0
Row 3	0	1	0
Row 4	0	0	1

Suppose that you want to select rows where Color is Red or Green. Using the bitmap index, the bitmap vectors for `Red` and `Green` can be OR-ed together.

→ [1, 0, 0] OR [0, 0, 1] = [1, 0, 1]

For each position with a `**1**`, the corresponding tuples in the table can then be fetched quickly.

Functional Index

For when you have a function to derive a value from the rows content.

```
CREATE INDEX index_name ON Employee(Salary +  
(Salary*Commission_pct));
```

```
SELECT First_name, Lname  
FROM Employee  
WHERE ((Salary*Commission_pct) + Salary ) > 15000;
```

References

Elmasri, R., & Navathe, S. B. (2017). Fundamentals of database systems, 7th ed. Harlow: Pearson Education.